Cluster Anchor Regularization to Alleviate Popularity Bias in Recommender Systems

Bo Chang
Google Inc.
Mountain View, CA, USA
bochang@google.com

Changping Meng
Google Inc.
Mountain View, CA, USA
changping@google.com

He Ma
Google Inc.
Mountain View, CA, USA
htm@google.com

Shuo Chang
Google Inc.
Mountain View, CA, USA
changshuo@google.com

Yang Gu
Google Inc.
Mountain View, CA, USA
greeness@google.com

Yajun Peng
Google Inc.
Mountain View, CA, USA
yajunpeng@google.com

Jingchen Feng
Google Inc.
Mountain View, CA, USA
jingchenfeng@google.com

Yaping Zhang
Google Inc.
Mountain View, CA, USA
yapingzhang@google.com

Shuchao Bi
Google Inc.
Mountain View, CA, USA
shuchaobi@google.com

Ed H. Chi
Google Inc.
Mountain View, CA, USA
edchi@google.com

Minmin Chen
Google Inc.
Mountain View, CA, USA
minminc@google.com

ABSTRACT

Recommender systems are essential for finding personalized content for users on online platforms. These systems are often trained on historical user interaction data, which collects user feedback on system recommendations. This creates a feedback loop leading to popularity bias; popular content is over-represented in the data, better learned, and thus recommended even more. Less popular content struggles to reach its potential audiences. Popularity bias limits the diversity of content that users are exposed to, and makes it harder for new creators to gain traction. Existing methods to alleviate popularity bias tend to trade off the performance of popular items. In this work, we propose a new method for alleviating popularity bias in recommender systems, called the cluster anchor regularization, which partitions the large item corpus into hierarchical clusters, and then leverages the cluster information of each item to facilitate transfer learning from head items to tail items. Our results demonstrate the effectiveness of the proposed method with offline analyses and live experiments on a large-scale industrial recommendation platform, where it significantly increases tail recommendation without hurting the overall user experience.

WWW '24 Companion, May 13–17, 2024, Singapore, Singapore

ACM ISBN 979-8-4007-0172-6/24/05.
https://doi.org/10.1145/3589335.3648312

CCS CONCEPTS

• **Information systems** → **Information retrieval**; **Web searching and information discovery**; **Personalization**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Recommender systems; popularity bias; hierarchical clustering.

ACM Reference Format:
Bo Chang, Changping Meng, He Ma, Shuo Chang, Yang Gu, Yajun Peng, Jingchen Feng, Yaping Zhang, Shuchao Bi, Ed H. Chi, and Minmin Chen. 2024. Cluster Anchor Regularization to Alleviate Popularity Bias in Recommender Systems. In *Companion Proceedings of the ACM Web Conference 2024 (WWW '24 Companion), May 13–17, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3589335.3648312

1 INTRODUCTION

Recommender systems are widely used in e-commerce, online music, movie streaming, and job recommendation services to match users with the right content. These systems typically rely on machine learning models that are trained on data about the past behavior of users already exposed to algorithmic recommendations; this creates a strong feedback loop [10]. Popular items are likely to be over-represented in the training data, causing them to be better learned and recommended than tail items, which in turn further boosts their popularity and over-representation in the training data. Some popular but engagement-deteriorating items gained unfair advantages and more-than-deserved visibility. Tail and fresh items, on the other hand, face significant challenges to reach their target audience. This phenomenon is commonly known as the *popularity bias* in recommender systems and has been extensively studied

[1–3, 14, 27, 30, 42, 43]. Throughout this paper, we refer to popular items as *head items*, and the others as *tail items*.

Alleviating popularity bias benefits all participants of the recommendation ecosystem: (i) **Users:** A more diverse set of recommendations spanning both head and tail items helps users discover new items or topics they might not have otherwise encountered, reducing content fatigue. (ii) **Content providers:** Helping tail items find their target audience enables content providers, often less established, to grow and gain traction on platforms. (iii) **Platforms:** A platform capable of indexing its entire inventory and recommending quality items regardless of popularity attracts more users and content providers, becoming more robust to changes. However, the aforementioned benefits can take a long time to manifest. Common strategies to reduce this bias focus on increasing the representation of tail items during training [8, 31]. However, these approaches often hurt the performance of popular (head) items [40].

To address these trade-offs, we propose the *cluster anchor regularization* technique. This method partitions the item corpus into hierarchical clusters and leverages this information to facilitate transfer learning from head items to tail items. Our technique offers broad applicability, improving tail item recommendations without sacrificing head item performance.

One key challenge in developing techniques to alleviate popularity bias lies in evaluation [3, 27, 42]. The Gini coefficient can be adopted to evaluate popularity bias in recommender systems [4, 5, 21, 41, 42]. However, the Gini coefficient has three major drawbacks when evaluating fast-iterating, large-scale systems. Firstly, it is dependent on sample size. Gains in small-traffic A/B testing may not accurately predict those of a full launch due to small-sample bias [13]. Secondly, it summarizes the system's overall inequality with a single number, making it less informative for holistic evaluation across different head/tail popularity buckets. Thirdly, in a real-world system with billions of items and millions of daily hot items, popularity bias naturally exists. A large Gini coefficient is insensitive to further improvement and can be easily obscured by noise. To overcome these limitations, we propose a novel corpus metric based on the effective size of the corpus – the number of unique items contributing to the top p percent of user interactions.

Together, we make the following contributions: (1) A hierarchical clustering algorithm for the item corpus, utilizing metadata and content information. (2) A regularization technique to enhance tail item recommendations without compromising head item performance. (3) A corpus metric specifically designed to measure improvements in reducing popularity bias. (4) Offline analyses providing insights and demonstrating the effectiveness of our proposed technique. (5) Large-scale live experiments demonstrating the regularization technique's benefits on a commercial platform, improving our corpus metric with neutral short-term user metrics.

2 RELATED WORK

2.1 Class Imbalance and Long-Tail Recommendation

Class imbalance challenges machine learning by biasing models towards majority classes. Methods to address this include: (1) oversampling/downsampling [20, 26, 37], (2) post-hoc decision correction [16, 19, 28], and (3) loss function modification [9, 12, 18, 31, 33, 38].

Long-tail item recommendation, where limited data exists for tail items, is a form of class imbalance. In recommender systems, the "long-tail" distribution means a few items get many interactions (head) while most get very few (tail) [30, 36]. This biases models towards head items [22, 35].

Approaches to improve this include post-processing recommendations [2, 42], loss function adjustments [8, 23, 24, 39], and new network architectures [5, 24, 40]. Clustering similar items is another method [30]. However, many of these approaches sacrifice popular item performance to help tail items [14, 36]. Our work aims to improve tail recommendations without harming popular ones via a tailored regularization term.

2.2 Clustering Algorithms

Clustering aims to group similar items. Popular algorithms include: *k-means clustering*, which iteratively assigns points to the closest centroids [25, 32]; *Density-based clustering*, which groups points in high-density regions [15]; and *Hierarchical clustering*, which forms nested clusters organized as a tree [7, 29].

In this paper, we propose a hierarchical clustering technique that is scalable, flexible, and balanced. Since cluster anchor regularization relies on high-quality item clusters for transfer learning, our proposed hierarchical clustering algorithm is crucial to its performance.

3 MOTIVATION

Recommender systems naturally exacerbate popularity bias since user behavior data reflects the system's tendency to favor well-established items. To quantify this bias, we analyzed a large-scale recommendation model. By ranking items by popularity and plotting the cumulative interaction percentage, we generated a Lorenz curve in Figure 1. This starkly illustrates the 'head-heavy' nature of the model, indicating that alleviating this issue could unlock the benefits for users, content providers, and platforms discussed in Section 1.

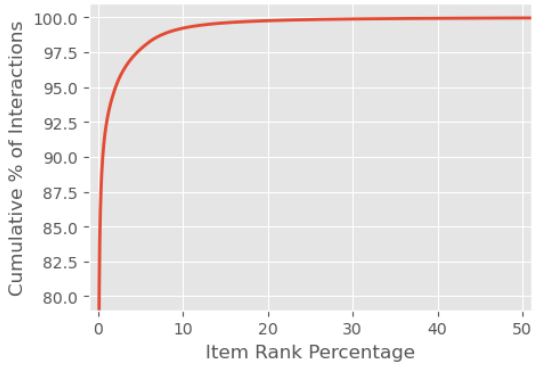


Figure 1: Illustration of the head-heavy issue in a large-scale recommender system. The figure shows the percentage of cumulative interactions attributed to a corresponding percentage of popular items.

One simple and intuitive solution we have tried in the past to alleviate the head-heavy issue is to expand the corpus and include

more tail items. In particular, on one of the recommendation models in the retrieval stage, the recommendable/output corpus was the top 1 million items based on popularity; we expanded the corpus to 3 million items in order to include more tail and fresh items and to help them get more exposure. This change was effective in improving freshness and corpus metrics (more details on corpus metrics in Section 6.1). However, there are two main drawbacks of this approach. First, we observe *diminishing returns* in expanding the corpus: further increasing the corpus size from 3 million to 10 million did not show much metric gain. This is because the model itself is popularity biased and fails to recommend the newly added tail items. Second, it incurs additional *computation costs*. For many recommendation models, item embeddings constitute a large portion of model parameters. Doubling or tripling the corpus size will inevitably increase training and serving costs. As a result, simply expanding the corpus is not sustainable and it motivates us to seek modeling innovations to alleviate popularity bias.

4 PROPOSED METHOD

To alleviate the head-heavy issue in recommender systems, we propose a technique called *cluster anchor regularization*. This technique leverages additional cluster information for each item to facilitate transfer learning from head items to tail items. Each item is assumed to be associated with a cluster assignment. For each cluster, we learn a *cluster anchor* based on head item representations within that cluster. The cluster anchor is then used to improve tail item representations in the same cluster. Section 4.1 provides an overview of possible cluster definitions and proposes a hierarchical clustering technique based on metadata and content of items. Section 4.2 describes the proposed cluster anchor regularization and how it is applied to a sequential recommendation model.

4.1 Hierarchical Clustering

An important prerequisite for cluster anchor regularization is a set of high-quality item clusters that allow for effective transfer learning between items within the same cluster. Such clusters can be generated based on metadata, content, engagement data, or a combination of these signals. There is a rich body of work on different clustering algorithms, offering various cluster characteristics (such as cluster size distribution and topic coherence). Many of these algorithms scale well to the magnitude of our problem.

In this work, we adopt a hierarchical clustering algorithm based on metadata and content. In particular,

(1) We first map a corpus of 2 billion items into 256-dimensional L2-normalized embeddings based on their metadata (title, hashtags, etc.) and content (frames and audio). These embeddings should not be confused with the item embeddings discussed in Section 4.2.
(2) We then construct a graph where each node represents an item and edge weights represent cosine similarities between embeddings generated in step (1).
(3) Next, we perform balanced graph partitioning through a distributed algorithm based on linear embedding [6], with the node weight representing the square root of the number of interactions received by the item.
(4) Finally, we recursively perform this graph partitioning step three times; each time, it is applied on top of the cluster centroids generated by the previous round of clustering. This leads to a 4-level tree structure over the corpus as shown in Figure 2.

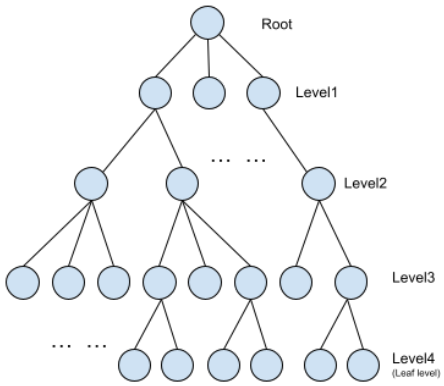


Figure 2: A 4-level tree structure containing 32, 768, 6K, and 48K nodes at each level respectively. The hierarchy represents the topic taxonomy that covers almost all items the recommender system would consider.

Based on this procedure, we generated a 4-level tree structure containing 32, 768, 6k, and 48k nodes respectively. Figure 2 provides an illustrative diagram. Figures 3 and 4 offer basic properties of the hierarchical clusters. In particular, Figure 3 shows the distribution of cluster sizes on each tree level[1]. One of the main constraints we enforced during cluster balancing is that the total number of interactions received by a cluster is roughly the same as others on the same tree level. Consequently, due to varying item popularity, cluster sizes (in terms of unique items) can differ significantly, as shown in Figure 3. However, the aggregated number of interactions received by all items within a cluster remains roughly the same across clusters on the same level. Figure 4 presents the distribution of median similarities between items and centroids of leaf clusters. For most leaf clusters, the median similarity is around 0.7, indicating good topic coherence.

The clustering approach we adopted exhibits several desirable properties:

- **Scalability:** The distributed algorithm scales effectively to large numbers of items. It takes approximately 16 hours to cluster the 2B training corpus into a 4-level tree with about 50K nodes. The tree structure and cluster centroid embeddings are frozen after initial training[2]. New items are assigned to clusters by finding their nearest leaf-level cluster centroid and associating them with corresponding clusters along the path to the root.
- **Flexible cluster granularity:** Each item is associated with 4 clusters (corresponding to the 4 tree levels). Higher-level clusters represent broader topics, while lower-level clusters

[1]Note that these statistics are produced based on all items in our corpus, not all of which are eligible for recommendation.
[2]In practice, the tree structure and cluster centroids could be updated periodically.

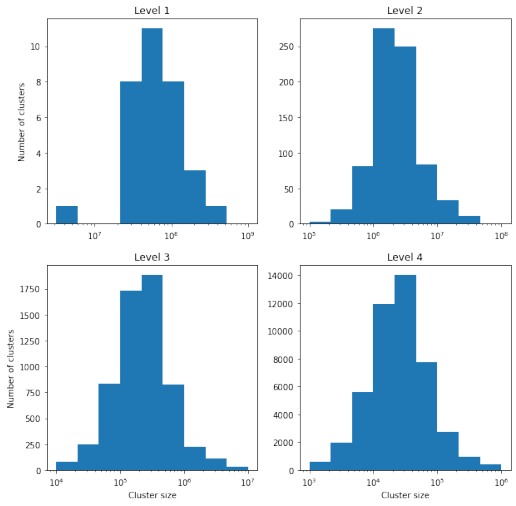


Figure 3: Histograms of cluster size distribution across the 4-level tree structure. Clusters at higher levels contain more items, while those at lower levels contain fewer items.

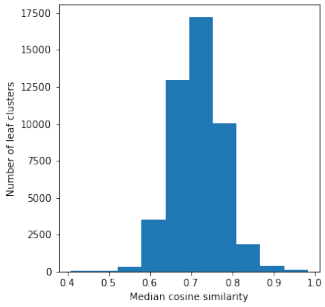


Figure 4: Histogram of median cosine similarities between items and their associated leaf cluster centroids.

represent narrower ones. This allows us to experiment with cluster anchor regularization at different tree levels to optimize performance (detailed in Section 6.4). The width and depth of the tree can be easily adjusted for different granularity requirements.

- **Balanced topic granularity within tree levels:** We formulate clustering as a graph partition problem, aiming for similar-sized partitions that minimize edge weights across partition boundaries. This balanced approach helps ensure clusters within the same tree level represent topics with a similar number of interactions. Each node at the same level contains items with topical coherence (bounded by a maximum distance to the centroid) and receives a similar amount of overall user engagement.

We emphasize that, while we used a specific clustering algorithm for demonstration purposes, the cluster anchor regularization approach proposed in this work is versatile and can be applied to clusters generated using various strategies.

4.2 Cluster Anchor Regularization

In this section, we present the proposed cluster anchor regularization technique. The objective is to perform knowledge transfer from *source* items to *target* items within the same cluster. The definitions of source and target items can be tailored to specific use cases. For example, head items can serve as the source and tail items as the target.

Let $\mathcal{I}$ be the item corpus on the platform. For each item $i \in \mathcal{I}$, a learnable n-dimensional embedding vector $\boldsymbol{v}_i \in \mathbb{R}^n$ is used as its representation. As discussed in Section 4.1, we further assume there exists a set of clusters $\mathcal{K} = \{1, 2, \ldots, K\}$, and for each item $i \in \mathcal{I}$ there is a unique cluster assignment, denoted by $k_i \in \mathcal{K}$. For each cluster $k \in \mathcal{K}$, there is also a learnable anchor vector $\boldsymbol{a}_k \in \mathbb{R}^n$, which can be regarded as the representation or centroid of the cluster. Let $\mathcal{S} \subseteq \mathcal{I}$ and $\mathcal{T} \subseteq \mathcal{I}$ be the sets of source and target items, respectively. Note that we do not require $\mathcal{S}$ and $\mathcal{T}$ to be disjoint. The goal is to transfer knowledge from the source items to the target ones within the same cluster.

The first component of the cluster anchor regularization is the *source anchor regularization*:

$$\mathcal{L}_{\mathcal{S}} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \|\boldsymbol{a}_{k_i} - \text{stop\_grad}(\boldsymbol{v}_i)\|^2, \tag{1}$$

where stop_grad$(\cdot)$ denotes the stop gradient operator that acts as the identity function in the forward pass and blocks the gradient flow in the backward pass, and k_i is the cluster annotation of the source item i. The source anchor regularization $\mathcal{L}_{\mathcal{S}}$ updates the anchors based on the source item representations from the same cluster. Since the source item embeddings are already well-learned, the stop gradient operator ensures that they are not adversely impacted by the regularization term.

From another perspective, the anchors are equivalent to the exponential moving average of the source item embeddings throughout training. This can be illustrated by a simplified example. Consider the mean squared error or L2 loss between an anchor $\boldsymbol{a}$ and an embedding $\boldsymbol{v}$, that is, $\ell = \frac{1}{2}\|\boldsymbol{a} - \boldsymbol{v}\|^2$, where $1/2$ is multiplied without loss of generality. The gradient of ℓ w.r.t $\boldsymbol{a}$ is $\nabla_{\boldsymbol{a}}\ell = \boldsymbol{a} - \boldsymbol{v}$. To learn the anchor $\boldsymbol{a}$ using gradient descent with a learning rate of $0 < \gamma < 1$, the update rule becomes

$$\boldsymbol{a} \leftarrow \boldsymbol{a} - \gamma\nabla_{\boldsymbol{a}}\ell = \boldsymbol{a} - \gamma(\boldsymbol{a} - \boldsymbol{v}) = (1 - \gamma)\boldsymbol{a} + \gamma\boldsymbol{v}. \tag{2}$$

It indicates that the cluster anchor $\boldsymbol{a}_k$ is the exponentially weighted moving average (EWMA) [17] of all source item embeddings from the same cluster k that the optimization algorithm has encountered during training.

The second component is the *target anchor regularization*. Its goal is to pull embeddings of the target items closer to the corresponding cluster anchors:

$$\mathcal{L}_{\mathcal{T}} = \frac{1}{|\mathcal{T}|} \sum_{j \in \mathcal{T}} \|\boldsymbol{v}_j - \text{stop\_grad}(\boldsymbol{a}_{k_j})\|^2. \tag{3}$$

where k_j is the cluster annotation of the target item j. This is to improve the target item representations without updating the anchors.

Similar to other regularization techniques, the cluster anchor regularization is added to the main loss function. The overall loss is

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \lambda_{\mathcal{S}}\mathcal{L}_{\mathcal{S}} + \lambda_{\mathcal{T}}\mathcal{L}_{\mathcal{T}}, \tag{4}$$

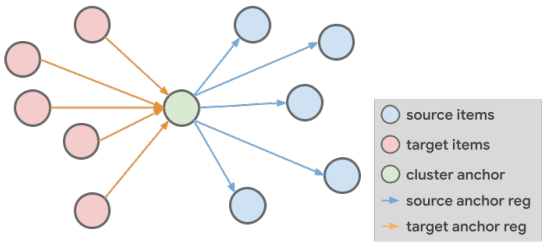


Figure 5: Illustration of cluster anchor regularization. The blue and red nodes represent source and target items from the same cluster, respectively. The green node denotes the corresponding cluster anchor. The blue arrow indicates the source anchor regularization ($\mathcal{L}_\mathcal{S}$) that pulls the cluster anchor toward the center of the source items; the orange arrow ($\mathcal{L}_\mathcal{T}$) pulls the target items toward the cluster anchor.

where $\mathcal{L}_{\text{main}}$ is the main loss function of the recommendation model; $\lambda_\mathcal{S} > 0$ and $\lambda_\mathcal{T} > 0$ are hyperparameters controlling the strengths of the source and target anchor regularizations, respectively. Figure 5 provides a graphical illustration of the cluster anchor regularization.

The proposed technique can be applied to any recommender models that involve item embeddings. In this work, we apply it to an RNN-based sequential recommendation model [11]. On a high-level, given a user representation $\boldsymbol{s} \in \mathbb{R}^n$ that summarizes the interaction history of the user, the model returns a softmax policy defined as

$$\pi(i|\boldsymbol{s}) = \frac{\exp(\boldsymbol{s}^\top \boldsymbol{v}_i / T)}{\sum_{j \in \mathcal{I}} \exp(\boldsymbol{s}^\top \boldsymbol{v}_j / T)}, \tag{5}$$

where $\boldsymbol{v}_i \in \mathbb{R}^n$ are item representations and $T > 0$ is a temperature that is normally set to 1. We refer interested readers to Chen et al. [11] for more details about the model. The proposed cluster anchor regularization is applied to the item representations $\boldsymbol{v}_i, \forall i \in \mathcal{I}$.

5 OFFLINE ANALYSIS

In this section, we report offline analysis results that offer insights into the mechanisms of the proposed cluster anchor regularization. Section 5.1 studies its effects on tail item coherence within the embedding space. Section 5.2 examines how quality metrics change for both head and tail items.

5.1 Improving Tail Item Coherence

We aim to study the effects of cluster anchor regularization on item embeddings, both quantitatively and qualitatively. With this regularization, tail embeddings are pulled toward their corresponding anchors. Consequently, we expect greater coherence among tail item embeddings within each cluster.

For analysis and illustration, we designate the top 10k items from our 4 million item corpus as "head" and the bottom 10k as "tail". Note that during learning, the source (head) and target (tail) sets $\mathcal{S}, \mathcal{T}$ are defined differently as detailed in Section 6.2. We use cosine distance to measure dissimilarity between two items i and j:

$$d_{\cos}(\boldsymbol{v}_i, \boldsymbol{v}_j) = 1 - \frac{\boldsymbol{v}_i^\top \boldsymbol{v}_j}{\|\boldsymbol{v}_i\| \|\boldsymbol{v}_j\|}. \tag{6}$$

Within each cluster, we compute the average pairwise cosine distance among the tail items.

Table 1: Average pairwise cosine distance among tail items grouped by clusters for the top-10 clusters. The average distance per cluster is consistently lower for the experiment model with cluster anchor regularization. This indicates that the embeddings of tail items in each cluster become more coherent.

Cluster ID	Baseline Model	Experiment Model	Difference
808	0.7422	0.6479	−12.72%
137	0.7288	0.6659	−8.62%
822	0.8340	0.7716	−7.48%
816	0.8575	0.7652	−10.76%
136	0.6342	0.4952	−21.92%
268	0.6897	0.6133	−11.08%
366	0.7867	0.6618	−15.89%
679	0.8497	0.6482	−23.71%
331	0.7985	0.6863	−14.05%
815	0.6133	0.3773	−38.49%
all tail items	0.9467	0.9064	−4.27%

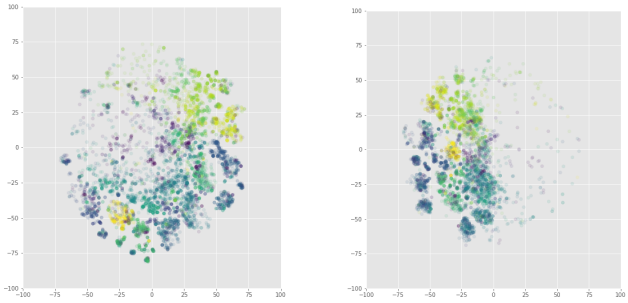
Table 1 compares the average distance among tail items within the same cluster between the baseline and experiment models. We focus on the top-10 clusters with the highest number of head items to ensure proper learning of cluster anchors. The results show a significant improvement in intra-cluster distance for the experiment model compared to the baseline. To address the possibility of a degenerate solution where all items are pushed to the same embedding, we also calculate the average pairwise cosine distance among all tail items (bottom row). This reference reveals a −4.27% change, indicating that the increased coherence of tail item embeddings within their clusters is not simply due to overall embedding shrinkage.

To further analyze the effects on tail item embeddings, we use t-distributed stochastic neighbor embedding (t-SNE) [34]. t-SNE is a technique for visualizing high-dimensional data by assigning each object a location in a lower-dimensional space (often 2D or 3D), preserving the relative similarities between objects. It's commonly used to visualize high-dimensional embeddings.

We fit separate t-SNE models for the baseline and experiment models and project the item embeddings into a two-dimensional space. Figure 6 visualizes the tail items, demonstrating that the cluster anchor regularization leads to more well-defined clusters. This qualitative result supports our previous quantitative findings. The colors in the figure represent level-1 hierarchical clusters.

5.2 Improving Accuracy on Tail Items

Recall that the proposed cluster anchor regularization is designed to transfer knowledge from head to tail items within the same cluster. Due to its specific stop-gradient design, we expect it to improve the recommendation model's performance on tail items without sacrificing performance on head items. In this section, we report offline evaluations on items grouped by their popularity rank (most popular item has rank 0). Items are divided into five buckets



(a) Baseline model. (b) Experiment model.

Figure 6: Qualitative analysis of tail item embeddings using t-SNE. The colormap represents level-1 hierarchical clusters. The figure demonstrates that cluster anchor regularization leads to more coherent cluster formations.

based on rank percentile: [0%, 1%), [1%, 25%), [25%, 50%), [50%, 75%), and [75%, 100%]. We compare the mean average precision (MAP@1) for each bucket between the baseline model and the model with cluster anchor regularization. Table 2 summarizes the results. Performance on the top 1% of items was slightly reduced, while performance significantly improved for all other buckets. This demonstrates that our proposed method alleviates popularity bias without substantially compromising performance on popular items.

Table 2: MAP@1 for items bucketed by rank. This demonstrates that cluster anchor regularization alleviates popularity bias without significantly sacrificing performance on popular items.

Item Rank	Baseline Model	Experiment Model	Difference
[0%, 1%)	0.0065	0.0057	−12.25%
[1%, 25%)	0.0062	0.0068	+10.24%
[25%, 50%)	0.0075	0.0079	+5.51%
[50%, 75%)	0.0070	0.0084	+21.28%
[75%, 100%]	0.0069	0.0078	+12.83%

6 LIVE EXPERIMENTS

In this section, we summarize the performance of our proposed method on a commercial recommendation platform serving billions of users.

6.1 Corpus Metric

One of the main challenges in developing techniques to alleviate popularity bias in recommendations lies in the choice of evaluation metrics. Improving tail items often leads to neutral or deteriorated short-term user metrics, while its long-term benefits take time to manifest. Therefore, we propose a corpus metric that can effectively measure improvements in mitigating popularity bias.

The Gini coefficient is a common inequality metric (0 = perfect equality, 1 = perfect inequality). However, it has several caveats, as we detail next. Thus, we designed a novel *corpus metric* to measure tail recommendation improvement. This corpus metric counts the number of most popular items contributing to the top percentages of overall consumption. It ranges from 1 to the corpus size: low values indicate dominance by a few popular items, while high values signify more even distribution. The corpus metric delta measures improvement in popularity bias mitigation:

$$\Delta_{\text{corpus}}@p = \frac{N_{\text{exp}}@p}{N_{\text{baseline}}@p} - 1, \tag{7}$$

where $N_{\text{exp}}@p$ and $N_{\text{baseline}}@p$ are the number of most popular items that contribute to the top $p\%$ of the consumption, for the experiment model and baseline model, respectively; p is the cutoff in $[0, 100]$. We report the corpus metric delta $\Delta_{\text{corpus}}@p$ as a function of the cutoff value p.

This proposed corpus metric has the following desiderata:

- **This metric is more sensitive than the Gini coefficient in a head-heavy system.** As we will show later in Section 6.3, our change (resulting in $\Delta_{\text{corpus}}@10 = 16\%$) translates to only a decrease of 0.005 in the Gini coefficient, which is barely distinguishable from measurement noise.
- **This metric is more robust to experiment traffic size.** We found the Gini coefficient to be relatively sensitive to experiment size due to potential outliers in small traffic experiments, and it suffers from small-sample bias [13]. Since the proposed corpus metric counts the number of items responsible for top percentages of consumption and compares relative numbers between control and experiment, it is much more robust in small traffic experiments.
- **By varying p from 0 to 100 in the proposed corpus metric, one can obtain a full spectrum of metrics and gain a holistic picture of overall system health.** For example, the metric $\Delta_{\text{corpus}}@10$ focuses on traffic/consumption concentration or dispersion among head items, while $\Delta_{\text{corpus}}@90$ allows for examination of the tail part of the corpus.

6.2 Experiment Setup

We conduct A/B experiments in a short-form content system serving billions of users to measure the benefits of cluster anchor regularization. A sequential recommender system [11] is built to retrieve hundreds of candidates from a corpus of millions of items upon each user request. The retrieved candidates, along with those returned by other sources, are then scored and ranked by a separate ranking system before presenting the top results to the user.

The A/B testing live experiments were run for 16 days. The control and experiment arms used the same sequential recommender model, with the exception of cluster anchor regularization (absent in the control, present in the experiment). During this period, both models were continuously updated every 4 hours using the latest live data as training data.

For the experiment model with cluster anchor regularization, hyperparameters are chosen as follows (more analyses on their effects and sensitivity are presented in Section 6.4):

- **Source examples:** Recall that source examples are high-quality items from which we transfer knowledge. We define them as items within a training batch having a completion

ratio greater than 150%, indicating a positive user experience[3].

- **Target examples:** Target examples are tail items whose embeddings we aim to improve. In each training step, we uniformly sample a set of items whose number of interactions ranks among the bottom 90% of the corpus; these are considered tail items. Conversely, the top 10% of the corpus are considered head items. The number of target examples is a hyperparameter.
- **Level of hierarchical clustering:** As discussed in Section 4.1, this level determines cluster granularity. We use level-2 clusters in the hierarchy, which contain 768 clusters.
- **Regularization strengths:** The strength of cluster anchor regularization is determined by the weights for source and target anchor regularizations ($\lambda_{\mathcal{S}}$ and $\lambda_{\mathcal{T}}$) defined in Equation 4. In our experiments, we set $\lambda_{\mathcal{S}} = 0.01$ and $\lambda_{\mathcal{T}} = 1$.

6.3 Experiment Results

User Metrics. We report the user metrics of the live experiments in Figure 7. The x-axis represents the date, and the y-axis shows the relative percentage difference between the experiment and control. We also report the mean and 95% confidence intervals for each metric.

Figure 7a shows that the overall enjoyment slightly decreased by −0.10% with a 95% confidence interval of [−0.22%, 0.03%]. This change is not statistically significant. Consumption decreased by −0.49% with a 95% confidence interval of [−0.67%, −0.32%] as shown in Figure 7b. better understand this change, we categorize consumption into *satisfied* and *unsatisfied* using a binary classification model trained on user satisfaction feedback data. Offline evaluation demonstrates this model has an accuracy of over 95%, indicating reliable categorization.

Figures 7c and 7d reveal the consumption drop is primarily driven by unsatisfied consumption, which decreased by −0.94% with a 95% confidence interval of [−1.26%, −0.62%]. This suggests improved user satisfaction as more tail and niche items are recommended. Conversely, satisfied consumption decreased by −0.21% with a 95% confidence interval of [−0.52%, 0.10%], a statistically insignificant change.

This improvement in user satisfaction is further supported by the metrics of like rate and number of dismissals. Like rate increased by +0.54% with a 95% confidence interval of [0.03%, 1.05%], and the number of dismissals decreased by −5.52% with a 95% confidence interval of [−11.57%, 0.52%]. We also analyzed candidates retrieved by the model with cluster anchor regularization. The like rate of these items increased by +2.29%, further indicating better user satisfaction.

Corpus Metric. Figure 8 shows the corpus metric delta $\Delta_{\text{corpus}}@p$ defined in Equation 7. On the x-axis is the cutoff value p, percentage of total consumption, ranging from 1 to 100; the y-axis corresponds to the relative change in the number of items contributed to the top p% of total consumption.

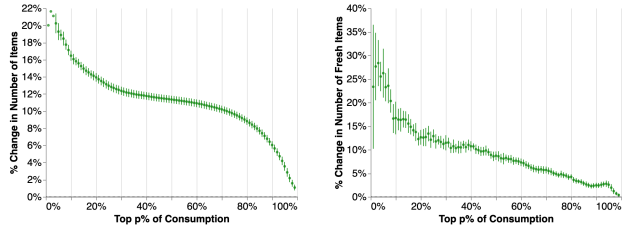
[3]In the context of short-form content, an item's completion ratio can exceed 100% if a user consumes it multiple times consecutively.

(a) Overall enjoyment (b) Consumption. (c) Satisfied consumption. (d) Unsatisfied consumption. (e) Like rate. (f) Number of dismissals.

Figure 7: Live experiment results for user metrics. The x-axis represents the date; the y-axis represents the relative difference (in percentage) between the experiment and control groups.

Figure 8a shows the corpus metric evaluated on all items. The number of items contributing to the top 10% and 90% of consumption increased by +16% and +6%, respectively. This indicates that consumption is more evenly distributed among both popular and tail items. We also computed the Gini coefficient for the change, which decreased from 0.88759 to 0.88208. As discussed in Section 6.1, our corpus metric is more sensitive, robust, and holistic compared to the single-number Gini coefficient.

Mitigating popularity bias can help fresh items gain traction and reach their target audiences. To illustrate this, we also evaluated the corpus metric on fresh items (i.e., items uploaded within 7 days). As shown in Figure 8b, the number of fresh items contributing to the top 10% and 90% of consumption increased by +16% and +2%, respectively, demonstrating the benefits for fresh items.



(a) For all items. (b) For fresh items.

Figure 8: Corpus metric $\Delta_{\text{corpus}}@p$ as a function of the cutoff value p, ranging from 1 to 100.

In addition to the proposed corpus metric, we also compare the cumulative distributions of item impressions between the experiment and control. This will provide an alternative perspective on the system's head-heaviness. In Figure 9, the x-axis represents the

item rank based on the number of interactions in the control, and the y-axis represents the cumulative percentage of impressions. The experiment model (shown in blue) favors less popular items, aligning with the conclusion from the corpus metric analysis. Furthermore, the unique number of recommended items also increased by +7.92%, suggesting that the cluster anchor regularization enables the retrieval model to reach a deeper corpus than the control model.

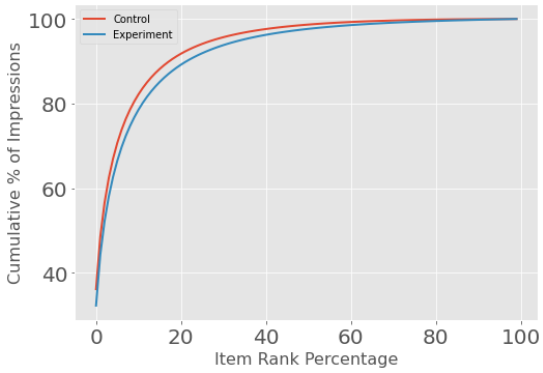


Figure 9: Cumulative distributions of item impressions in the experiment and control. The item rank on the x-axis is computed based on the number of interactions in control.

Computation Cost. The computational cost of adding cluster anchor regularization is negligible. While it introduces a few additional computations, these costs are insignificant relative to the overall model training process. Similar to other regularization techniques, cluster anchor regularization is enabled only during training and does not incur any additional serving cost.

6.4 Sensitivity Analysis

In this section, we discuss different model designs and the tuning of important hyperparameters[4].

Source Examples. In our experiment, we use the completion ratio to select high-quality items as source examples. This threshold balances the trade-off between bias and variance of the cluster anchors. A higher threshold leads to lower bias but higher variance. Conversely, a lower threshold corresponds to higher bias and lower variance.

We experimented with completion ratios of 90%, 150%, and 200%. Table 3 summarizes the comparison. On corpus metrics, models using completion ratios of 90% or 150% outperformed the baseline: +17% on Δ_{corpus}@10 and +7% on Δ_{corpus}@90. These models also performed better than the one with a ratio of 200%. Based on the user preference metric, a threshold of 150% was optimal.

Target Examples. We also experimented with an alternative definition of target examples: instead of sampling exclusively from tail items, we uniformly sampled items from the entire corpus. This, however, led to a significant drop in user metrics (e.g., a −0.15% reduction in overall enjoyment with a 95% confidence interval of [−0.22%, −0.08%]). We hypothesized that this decline was due to the target anchor regularization negatively impacting the representation of head items. Our final experimental model randomly samples target examples from the bottom 90% of the corpus, updating only the embeddings of tail items through the target anchor regularization. This approach avoided compromising head recommendation quality.

[4]To limit the impact on user experience, only a small percentage of traffic is allocated for live experiments simultaneously. As a result, some hyperparameters are tuned in different rounds of experiments. While the trends are consistent across rounds, the precise numerical values may differ slightly.

Table 3: Comparison across different completion ratios used as the source example threshold. We report user metrics (like rate), corpus metric delta@10%, and corpus metric delta@90%.

Completion Ratio	Like Rate	Δ_{corpus}@10	Δ_{corpus}@90
90%	+0.48% [−0.12%, +1.08%]	+17%	+7%
150%	+0.54% [+0.03%, +1.05%]	+16%	+7%
200%	+0.32% [−0.18%, +0.82%]	+11%	+6%

Level of Hierarchical Clustering. Our hierarchical clustering algorithm provides four levels of cluster granularity, with 32, 768, 6K, and 48K clusters respectively. We tested the cluster anchor regularization with different cluster levels in live experiments. All cluster levels led to similar user metrics, but the corpus metric varied significantly. The gain from using level-2 clusters (our experimental model) is shown in Figure 8a. We did not experiment with level-1 clusters, as they are too coarse to provide meaningful information for tail item improvement. For level-3 and level-4 clusters, the corpus metric delta Δ_{corpus}@10 is 33% and 50% lower than that of level-2, respectively. When finer-grained clusters are used, it becomes more difficult to find a sufficient number of representative source examples to guide the movement of tail items.

Regularization Strengths. We also experimented with different regularization strengths. In the first experimental round, we set $\lambda_{\mathcal{S}} = \lambda_{\mathcal{T}} \in \{1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$. We observed that $\lambda_{\mathcal{S}} = \lambda_{\mathcal{T}} = 10^{-2}$ achieved the best corpus metric with a neutral impact on user metrics. In the second round, we further fine-tuned the weights, allowing $\lambda_{\mathcal{S}} \neq \lambda_{\mathcal{T}}$, and found that $\lambda_{\mathcal{S}} = 10^{-2}$ and $\lambda_{\mathcal{T}} = 1$ yielded the greatest improvement in user metrics.

7 CONCLUSION

In summary, we proposed cluster anchor regularization, a novel approach to address popularity bias in recommender systems through cluster-based transfer learning. This versatile technique can be integrated into any recommender model leveraging learned item embeddings. Using an RNN-based sequential recommendation model, we demonstrated significant improvements in both prediction accuracy and tail item embedding quality. Live experiments further validated our approach, showing substantial gains in corpus metrics without sacrificing short-term user metrics. This success highlights the regularization's design, ensuring unidirectional transfer learning from head to tail items. Our work provides a powerful and practical solution for effective long-tail recommendation in large-scale industrial recommender systems.

REFERENCES

[1] Himan Abdollahpouri. 2019. Popularity bias in ranking and recommendation. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*. 529–530.

[2] Himan Abdollahpouri, Robin Burke, and Bamshad Mobasher. 2019. Managing popularity bias in recommender systems with personalized re-ranking. In *The thirty-second international flairs conference*.

[3] Himan Abdollahpouri, Masoud Mansoury, Robin Burke, and Bamshad Mobasher. 2020. The connection between popularity bias, calibration, and fairness in recommendation. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 726–731.

[4] Himan Abdollahpouri, Masoud Mansoury, Robin Burke, Bamshad Mobasher, and Edward Malthouse. 2021. User-centered evaluation of popularity bias in recommender systems. In *Proceedings of the 29th ACM Conference on User Modeling, Adaptation and Personalization*. 119–129.

[5] Arda Antikacioglu and R Ravi. 2017. Post processing recommender systems for diversity. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 707–716.

[6] Kevin Aydin, MohammadHossein Bateni, and Vahab Mirrokni. 2016. Distributed balanced partitioning via linear embedding. In *Proceedings of the Ninth ACM International Conference on Web Search and Data Mining*. 387–396.

[7] MohammadHossein Bateni, Soheil Behnezhad, Mahsa Derakhshan, Hajjaghayi MohammadTaghi, Raimondas Kiveris, Silvio Lattanzi, and Vahab Mirrokni. 2017. Affinity Clustering: Hierarchical Clustering at Scale. In *Proceedings of 31st Conference on Neural Information Processing Systems (NIPS 2017)*.

[8] Alex Beutel, Ed H Chi, Zhiyuan Cheng, Hubert Pham, and John Anderson. 2017. Beyond globally optimal: Focused learning for improved recommendations. In *Proceedings of the 26th International Conference on World Wide Web*. 203–212.

[9] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arechiga, and Tengyu Ma. 2019. Learning imbalanced datasets with label-distribution-aware margin loss. *Advances in neural information processing systems* 32 (2019).

[10] Allison JB Chaney, Brandon M Stewart, and Barbara E Engelhardt. 2018. How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. In *Proceedings of the 12th ACM conference on recommender systems*. 224–232.

[11] Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, and Ed H Chi. 2019. Top-k off-policy correction for a REINFORCE recommender system. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. 456–464.

[12] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. 2019. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9268–9277.

[13] George Deltas. 2003. The small-sample bias of the Gini coefficient: results and implications for empirical research. *Review of economics and statistics* 85, 1 (2003), 226–234.

[14] Marcos Aurélio Domingues, Fabien Gouyon, Alípio Mário Jorge, José Paulo Leal, João Vinagre, Luís Lemos, and Mohamed Sordo. 2013. Combining usage and content in an online recommendation system for music in the long tail. *International Journal of Multimedia Information Retrieval* 2, 1 (2013), 3–13.

[15] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise.. In *kdd*, Vol. 96. 226–231.

[16] Tom Fawcett and Foster J Provost. 1996. Combining Data Mining and Machine Learning for Effective User Profiling. In *KDD*.

[17] J Stuart Hunter. 1986. The exponentially weighted moving average. *Journal of quality technology* 18, 4 (1986), 203–210.

[18] Muhammad Abdullah Jamal, Matthew Brown, Ming-Hsuan Yang, Liqiang Wang, and Boqing Gong. 2020. Rethinking class-balanced methods for long-tailed visual recognition from a domain adaptation perspective. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7610–7619.

[19] Bingyi Kang, Saining Xie, Marcus Rohrbach, Zhicheng Yan, Albert Gordo, Jiashi Feng, and Yannis Kalantidis. 2020. Decoupling Representation and Classifier for Long-Tailed Recognition. In *International Conference on Learning Representations*.

[20] M KUBAT. 1997. Addressing the curse of imbalanced training stes: one-sided selection. In *Proceedings of the 14th International Conference on Machine Learning, 1997*. 79–186.

[21] Jurek Leonhardt, Avishek Anand, and Megha Khosla. 2018. User fairness in recommender systems. In *Companion Proceedings of the The Web Conference 2018*. 101–102.

[22] Jingjing Li, Ke Lu, Zi Huang, and Heng Tao Shen. 2017. Two birds one stone: on both cold-start and long-tail recommendation. In *Proceedings of the 25th ACM international conference on Multimedia*. 898–906.

[23] Allen Lin, Jianling Wang, Ziwei Zhu, and James Caverlee. 2022. Quantifying and mitigating popularity bias in conversational recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 1238–1247.

[24] Siyi Liu and Yujia Zheng. 2020. Long-tail session-based recommendation. In *Fourteenth ACM conference on recommender systems*. 509–514.

[25] J MacQueen. 1967. Classification and analysis of multivariate observations. In *5th Berkeley Symp. Math. Statist. Probability*. University of California Los Angeles LA USA, 281–297.

[26] Dhruv Mahajan, Ross Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe, and Laurens Van Der Maaten. 2018. Exploring the limits of weakly supervised pretraining. In *Proceedings of the European conference on computer vision (ECCV)*. 181–196.

[27] Elisa Mena-Maldonado, Rocío Cañamares, Pablo Castells, Yongli Ren, and Mark Sanderson. 2021. Popularity bias in false-positive metrics for recommender systems evaluation. *ACM Transactions on Information Systems (TOIS)* 39, 3 (2021), 1–43.

[28] Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. 2021. Long-tail learning via logit adjustment. In *International Conference on Learning Representations*.

[29] Fionn Murtagh and Pedro Contreras. 2012. Algorithms for hierarchical clustering: an overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 2, 1 (2012), 86–97.

[30] Yoon-Joo Park and Alexander Tuzhilin. 2008. The long tail of recommender systems and how to leverage it. In *Proceedings of the 2008 ACM conference on Recommender systems*. 11–18.

[31] Jiawei Ren, Cunjun Yu, Xiao Ma, Haiyu Zhao, Shuai Yi, et al. 2020. Balanced meta-softmax for long-tailed visual recognition. *Advances in neural information processing systems* 33 (2020), 4175–4186.

[32] D. Sculley. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International World Wide Web Conference (WWW 2010)*.

[33] Jingru Tan, Changbao Wang, Buyu Li, Quanquan Li, Wanli Ouyang, Changqing Yin, and Junjie Yan. 2020. Equalization loss for long-tailed object recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11662–11671.

[34] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).

[35] Maksims Volkovs, Guangwei Yu, and Tomi Poutanen. 2017. Dropoutnet: Addressing cold start in recommender systems. *Advances in neural information processing systems* 30 (2017).

[36] Hongzhi Yin, Bin Cui, Jing Li, Junjie Yao, and Chen Chen. 2012. Challenging the Long Tail Recommendation. *Proceedings of the VLDB Endowment* 5, 9 (2012).

[37] Xi Yin, Xiang Yu, Kihyuk Sohn, Xiaoming Liu, and Manmohan Chandraker. 2019. Feature Transfer Learning for Face Recognition with Under-Represented Data. In *In Proceeding of IEEE Computer Vision and Pattern Recognition*.

[38] Xiao Zhang, Zhiyuan Fang, Yandong Wen, Zhifeng Li, and Yu Qiao. 2017. Range loss for deep face recognition with long-tailed training data. In *Proceedings of the IEEE International Conference on Computer Vision*. 5409–5418.

[39] Yin Zhang, Derek Zhiyuan Cheng, Tiansheng Yao, Xinyang Yi, Lichan Hong, and Ed H Chi. 2021. A model of two tales: Dual transfer learning framework for improved long-tail item recommendation. In *Proceedings of the web conference 2021*. 2220–2231.

[40] Yin Zhang, Ruoxi Wang, Derek Zhiyuan Cheng, Tiansheng Yao, Xinyang Yi, Lichan Hong, James Caverlee, and Ed H Chi. 2022. Empowering Long-tail Item Recommendation through Cross Decoupling Network (CDN). *arXiv preprint arXiv:2210.14309* (2022).

[41] Renjie Zhou, Samamon Khemmarat, and Lixin Gao. 2010. The impact of YouTube recommendation system on video views. In *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*. 404–410.

[42] Ziwei Zhu, Yun He, Xing Zhao, and James Caverlee. 2021. Popularity bias in dynamic recommendation. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2439–2449.

[43] Ziwei Zhu, Yun He, Xing Zhao, Yin Zhang, Jianling Wang, and James Caverlee. 2021. Popularity-opportunity bias in collaborative filtering. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. 85–93.